Equipo: 6

Integrantes: Emmanuel Pulido, David Páez, Gilberto Anaya

Materia: Desarrollo en la Nube

Fecha: 29 de octubre de 2026

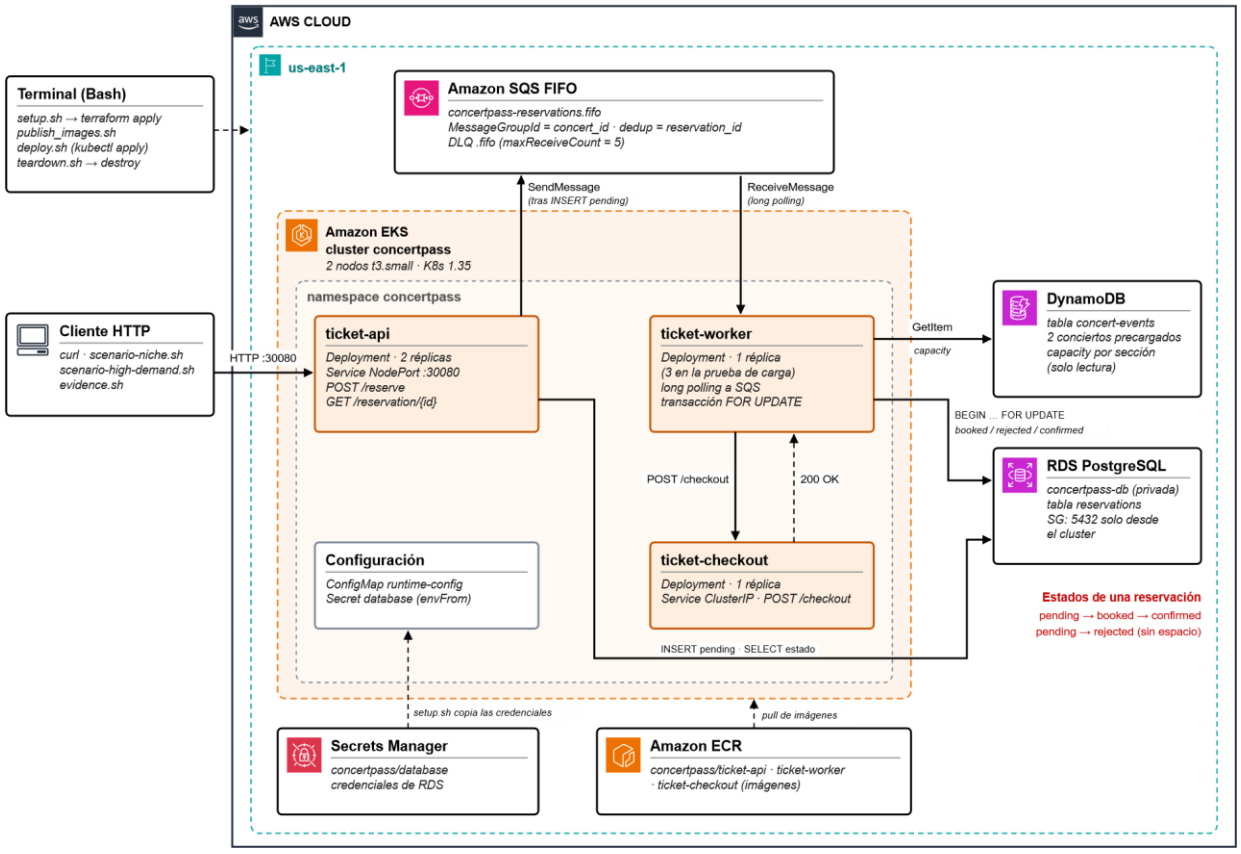
Descripción del proyecto y diagrama de arquitectura

Concertpass es un sistema de reservación de boletos para conciertos que debe recibir picos de demanda sin vender más del aforo disponible. Son tres microservicios en Python, cada uno con su Dockerfile, cuyas imágenes se publican en Amazon ECR y se despliegan como Deployments en un cluster de Amazon EKS con dos nodos.

La infraestructura se levanta con setup.sh, que ejecuta Terraform para crear el cluster, RDS PostgreSQL, una cola SQS FIFO con su DLQ, la tabla de DynamoDB con dos conciertos precargados, el secreto de Secrets Manager y los repositorios de ECR. Después copia las credenciales de RDS a un Secret de Kubernetes y la demás configuración a un ConfigMap; los servicios la leen de variables de entorno. teardown.sh destruye todos los recursos.

El cliente envía POST /reserve a ticket-api, expuesto con un Service NodePort. ticket-api guarda la reservación en RDS como pending, la encola en SQS con concert_id como MessageGroupId y regresa el reservation_id. ticket-worker consume la cola: lee la capacidad de la sección en DynamoDB y, en una transacción con FOR UPDATE, suma los boletos booked y confirmed del mismo concierto y tipo. Si hay espacio marca la reservación como booked y llama a POST /checkout de ticket-checkout (Service ClusterIP), que simula el procesador de pagos y responde 200 OK; entonces la marca como confirmed. Si no hay espacio hace ROLLBACK y la marca como rejected. El estado se consulta con GET /reservation/{id}.

Validamos dos escenarios con scripts: nicho, con 4 reservaciones que terminan confirmadas, y alta demanda, con 60 reservaciones VIP para 50 lugares y 3 réplicas del worker, donde las sobrantes quedan rejected.



1. ¿Para qué sirve SQS FIFO con MessageGroupId y cómo se usa en esta actividad?

Una cola FIFO entrega los mensajes en el orden en que se enviaron y descarta duplicados con el mismo MessageDeduplicationId durante cinco minutos. El orden se garantiza dentro de cada MessageGroupId: mientras un mensaje de un grupo está en proceso, SQS no entrega los siguientes mensajes de ese grupo a otro consumidor, pero los grupos distintos sí avanzan en paralelo. En Concertpass, ticket-api envía cada reservación con concert_id como MessageGroupId y reservation_id como MessageDeduplicationId, de modo que las reservaciones de un concierto se atienden en orden de llegada (el primero que reserva obtiene el boleto) y un reintento del envío no genera una reservación duplicada. El worker borra el mensaje solo después de procesarlo y, si falla, deja en la cola el resto de los mensajes de ese concierto para no alterar el orden, y después de cinco intentos el mensaje pasa a la DLQ.

2. ¿Qué problema resuelve FOR UPDATE y cómo garantiza consistencia? ¿Cuál es el trade-offentre concurrencia y consistencia?

Resuelve una condición de carrera de tipo leer-verificar-escribir: sin bloqueo, dos workers podrían leer al mismo tiempo que queda un boleto, aprobar cada uno su reservación y vender más del aforo. FOR UPDATE bloquea las filas leídas hasta el COMMIT o ROLLBACK, así que otro worker que necesita las mismas filas espera y, al continuar, ve la suma ya actualizada. Como PostgreSQL no permite FOR UPDATE sobre una agregación, el worker bloquea en una subconsulta las filas del mismo concierto y tipo (incluidas las pending, para que siempre haya una fila en común aunque todavía no haya ventas) y suma afuera las booked y confirmed. La verificación y el cambio a booked ocurren en la misma transacción; si no hay espacio, el ROLLBACK libera el bloqueo y una transacción separada cambia a rejected solo esa fila. Cada UPDATE exige además el estado anterior (WHERE status = 'pending'), por lo que procesar dos veces el mismo mensaje no vende dos veces.

El trade-off es que la consistencia se paga con concurrencia. Mientras un worker tiene el bloqueo, los demás que buscan la misma sección esperan, así que las reservaciones de un concierto popular se procesan prácticamente en serie (SQS FIFO ya las serializa por concert_id). Más réplicas del worker no aceleran un solo concierto, ayudan cuando hay varios conciertos o cuando una réplica falla. ticket-api sí escala horizontalmente porque solo inserta y encola, de modo que absorbe la ráfaga y la cola guarda el exceso. El cuello de botella real es la lógica de inventario (lectura, bloqueo y escritura en RDS). Aceptamos más latencia a cambio de no sobrevender, que es el error que el negocio no puede permitirse.

3. ¿Qué beneficios te da usar EKS en este caso?

EKS nos da un plano de control de Kubernetes administrado por AWS, disponible y compatible con kubectl, sin instalar ni mantener los masters; los nodos los administra un managed node group. Sobre eso, Kubernetes nos permite describir los tres servicios en manifiestos versionados en el repositorio, escalar cada uno por separado (2 réplicas de ticket-api y el worker escalado a 3 con kubectl scale durante la prueba de carga) y recrear automáticamente un pod que falla. El DNS interno permite que el worker llame a ticket-checkout por nombre, y los tipos de Service separan lo público (NodePort para ticket-api) de lo interno (ClusterIP para checkout). La configuración entra por ConfigMap y Secret, así que la misma imagen de ECR funciona en cualquier entorno. Además se integra con la red de AWS, donde los pods reciben IPs de la VPC y el security group de RDS acepta conexiones solo desde el cluster. La contraparte es el costo del plano de control y de los nodos mientras el cluster existe, por eso el ciclo termina con teardown.sh.

Anexos

Repositorio de GitHub: https://github.com/Dakewer/concertpass

Video demostrativo: